

Expense Voucher Management System (EVMS)

System Design & Architecture — Condensed Summary
Waterfall SDLC · Full-Stack Web Application

Document Version	1.0 (Condensed)
Date	31 August 2026
Prepared For	ABC Company — Prachay Securities
Stack	React 19 + Spring Boot 3.3 + MySQL 8

EVMS digitises ABC Company's paper-based expense voucher process into a role-based digital workflow (Employee / Director / Accounts) with full audit trail. Built with a single-developer Waterfall SDLC due to fixed, well-defined requirements.

1. Architecture Overview

Three-tier architecture, each layer built and verified sequentially per Waterfall phases (Requirements → Design → Detailed Design → Implementation → Testing → Deployment):

Tier	Technology
Presentation	React 19 SPA · Vite · React Router 7 · Axios (REST/JSON over HTTPS)
Business Logic	Spring Boot 3.3 · Java 17 · Spring Security (JWT) · Hibernate/JPA 6
Data	MySQL 8 — tables: users, vouchers, voucher_counter · indexes & FKs

Other stack details: BCrypt (10 rounds) password hashing, JWT 0.12.5 for token handling, Maven build, vanilla CSS ('ledger & paper voucher' design, no UI framework).

2. Requirements Summary

Functional

- JWT login for all roles; role-specific dashboards with KPI summaries
- Employees create/edit/delete their own **DRAFT** vouchers (auto-numbered EV-{year}-{seq})
- Submit (Employee, requires signature) → Approve/Reject (Director, requires signature or reason)
- Employees see only their own vouchers; Director/Accounts see all
- Search, filter, sort by number, employee, department, category, status, date and amount range
- APPROVED/REJECTED are terminal; expense date can't be future-dated; amount must be > 0

Non-Functional

- Performance: API responses <500ms; file uploads capped at 2MB/file, 5MB/request
- Security: BCrypt hashing, JWT expiry 8h, RBAC enforced server-side (not just hidden in UI)
- Usability: responsive on desktop browsers (≥1024px)
- Portability: standard SQL + JPA ORM allows migration from MySQL to PostgreSQL

3. User Roles & Permissions

Action	Employee	Director	Accounts
Create / Edit / Delete DRAFT voucher	✓	✗	✗
Submit voucher	✓	✗	✗
Approve / Reject voucher	✗	✓	✗
View own vouchers only	✓	✗	✗
View all vouchers (org-wide)	✗	✓	✓
Search / filter / sort	own	all	all

4. Database Design

users (1) — (N) **vouchers** via `employee_id` FK; `vouchers.approved_by` also FKs to `users`. Key fields:

Table	Key Columns
users	id (PK), name, email (unique), password_hash, role (enum), department, created_at
vouchers	id (PK), voucher_number (unique), employee_id (FK), department, voucher_date, expense_date, expense_title, category
voucher_counter	year (PK), seq — dedicated counter table for atomic voucher numbering (EV-{year}-{0001..9999}), avoids race conditions

Indexes: `idx_vouchers_employee` (`employee_id` — fast per-employee lookup), `idx_vouchers_status` (`status` — fast pending/KPI filtering).

5. API Design

Base URL `/api`. All endpoints except `/auth/login` and `/health` require `Authorization: Bearer <JWT>`.

Method	Endpoint	Role	Description
POST	<code>/auth/login</code>	Public	Returns JWT + user object
GET	<code>/auth/me</code>	Any	Current user from JWT
POST	<code>/vouchers</code>	Employee	Create DRAFT voucher (auto-numbered)
GET	<code>/vouchers</code>	Any	List (role-scoped, filtered, sorted, paginated)
GET	<code>/vouchers/{id}</code>	Any	Single voucher (403 if not own, for Employee)
PUT	<code>/vouchers/{id}</code>	Employee (owner)	Update DRAFT fields
DELETE	<code>/vouchers/{id}</code>	Employee (owner)	Delete DRAFT only
POST	<code>/vouchers/{id}/submit</code>	Employee (owner)	DRAFT→SUBMITTED (signature upload, multipart)
POST	<code>/vouchers/{id}/approve</code>	Director	SUBMITTED→APPROVED (signature upload, multipart)
POST	<code>/vouchers/{id}/reject</code>	Director	SUBMITTED→REJECTED (JSON reason)
GET	<code>/vouchers/dashboard/summary</code>	Any	Role-specific KPI JSON

GET `/vouchers` query params: `voucherNumber`, `employeeName`, `department`, `category`, `status`, `dateFrom/dateTo`, `amountMin/amountMax`, `sortBy`, `sortDir`, `page`, `limit` (default 10/page).

6. Authentication & Security

- Stateless JWT auth — no server-side sessions; token stored client-side, sent as Bearer header
- BCrypt (10 rounds) password hashing — never stored/transmitted in plain text
- `JwtAuthenticationFilter` validates every request and populates the `SecurityContext`
- Method-level RBAC via `@PreAuthorize("hasRole(...)")` on controllers
- Row-level scoping: employees restricted to own vouchers via JPA Specification (SQL WHERE), not UI-only
- CORS allowed for `localhost:5173` (dev); CSRF disabled (appropriate for stateless REST)

7. Voucher State Machine

Transition	Rule
DRAFT → SUBMITTED	Owning Employee only; requires employee signature upload
SUBMITTED → APPROVED	Director only; requires director signature upload
SUBMITTED → REJECTED	Director only; requires non-empty rejection reason
APPROVED / REJECTED	Terminal — no further transitions permitted
Edit / Delete	DRAFT vouchers only, owning Employee only

8. Implementation Notes

Backend

- Layered MVC: Controller → Service → Repository, with a DTO layer decoupling API contracts from JPA entities
- Spring Data JPA repositories; Specification pattern for dynamic voucher filtering
- Global @ControllerAdvice exception handler standardises error JSON: 400 (validation), 401 (auth), 403 (access denied), 404 (not found), 409 (invalid state transition)
- Standard package layout: config / controller / dto (request, response) / exception / model (entities + enums) / repository / security / service

Frontend

- SPA with React Router; role-prefixed routes (/employee/*, /director/*, /accounts/*) guarded by ProtectedRoute
- AuthContext (Context API) holds auth state; centralised Axios instance auto-attaches JWT
- Custom vanilla CSS design system (~15KB), no UI framework dependency
- Signature upload via HTML file input + FormData (multipart/form-data)

9. Testing Strategy

Level	Tools	Coverage
Unit	JUnit 5, Spring Boot Test	VoucherService (state transitions, validation), AuthService (login)
Integration	Spring MockMvc, @SpringBootTest	All REST endpoints: CRUD, workflow, auth, error responses
End-to-End	Manual (browser)	Full voucher lifecycle for each role

16 documented test scenarios cover login (valid/invalid), voucher CRUD validation (future date, non-positive amount), submit/approve/reject with and without required signature/reason, RBAC violations (403s), invalid state transitions (409), filtered list pagination, and role-specific dashboard shapes.

10. Deployment

Prerequisites: JDK 17+, Node.js 18+, MySQL 8 (db: evms_db), Maven 3.x

Backend:

```
cd backend && ./mvnw.cmd spring-boot:run → localhost:4000
```

Frontend:

```
cd frontend && npm install && npm run dev → localhost:5173
```

Role	Email	Password
Employee	employee@evms.test	password123
Director	director@evms.test	password123

Role	Email	Password
Accounts	accounts@evms.test	password123

Production hardening (not yet implemented): replace ddl-auto=update with Flyway/Liquibase migrations; move signature storage to cloud object storage (S3/Blob); externalise JWT secret via env var; terminate TLS via reverse proxy; add an admin user-provisioning endpoint; add Spring Actuator for monitoring.

11. Assumptions & Constraints

- Users are provisioned internally via data.sql seed — no public registration screen
- Signature images live on local disk for v1.0 (cloud storage planned for production)
- Targets modern desktop browsers ($\geq 1024\text{px}$); mobile-responsive design out of scope for v1.0
- Designed for modest concurrent load (< 100 users); no horizontal scaling in v1.0
- Fixed stack constraints: Java 17 + Spring Boot backend, React frontend, MySQL database, JWT-only auth, no external UI framework

Condensed from the full 27-page System Design & Architecture document (v1.0, August 2026). Full API DTO shapes, complete file/package trees, DFDs and the glossary are omitted here for brevity — available in the source document.